Introduction

- **What is Database Management System?**

A database management system (DBMS) is a collection of interrelated data and a set of programs to access those data. The collection of data, usually referred to a **database**, contains information relevant to an enterprise.

- **Goals of a DBMS**

1. The primary goal of a DBMS is to provide an environment that is both convenient and efficient for people in retrieving and storing information.

2. Database systems are designed to manage large bodies of information. Management of data involves both defining structures for storage of information and providing mechanisms for manipulation of information.

3. The database system must ensure the safety and security of the information stored, despite system crashes or attempts at unauthorized access.

4. The system must avoid possible anomalous results if data are to be shared among several users.

1.1 Database System Applications

Databases are widely used. Some representative applications are:

- **Enterprise Information**

Sales: For customer, product and purchase information.

Accounting: For payment, receipts, account balances, assets and other accounting information.

Human resources: For information about employees, salaries, payroll taxes and benefits and for generation of pay checks.

Manufacturing: For management of supply chains and for tracking production of items in factories, inventories of items in warehouses/stores and order for items.

On-line retailers: For sales data model above plus on-line order tracking, generation of recommendation lists and maintenance of on-line product evaluation.

- **Banking and Finance**

Banking: For customer information, accounts, loans and banking transactions.

Credit card transaction: For purchases on credit card, monthly statement generation

Finance: For storing information about holdings, sales, and purchases of financial instruments such as stocks and bonds; also for storing real-time market data to enable on-line trading by customers and automated trading by the firm.

- **Universities:** For student information, courses and grades (education management).

- **Airlines/Railways/Road Transport:** For ticket reservation, schedules and routes.

- **Telecommunication:** For keeping records of call made, generating monthly bills, maintaining balances on prepaid calling cards, storing information about the communication networks.

- **Internet:** Internet is the major repository of database.

❖ Some Database Management Systems

- **Commercial (Large Scale) Database Management Systems**

1. Oracle – Oracle 8i, Oracle9i, Oracle 10g, Oracle 11g, Oracle 12c
2. Microsoft SQL Server
3. IBM DB2/DB2UDB
4. Informix (Now own by IBM)
5. Sybase
6. MySQL (free/public domain)
7. Ingress (free/public domain)
8. PostgreSQL (free/public domain)

- **Small Scale / Personal use Database Management Systems**

1. Microsoft Access
2. FoxPro
3. dBase
4. Oracle 10g / 11g Express Edition

1.2 Purpose of Database Systems (Database Management Systems Vs File Systems)

Ordinary file system has a number of major drawbacks:

1. Data redundancy and inconsistency

- Multiple file formats, duplication of information in different files leading to a higher storage and access cost. Again various copies of same data may no longer agree leading to data inconsistency.

2. Difficulty in accessing data

- Need to write a new program to carry out each new task.

3. Data isolation

- Because data are scattered in various files with different formats, writing new application programs to retrieve the appropriate data is difficult.

4. Integrity problems

- The data values stored in the database must satisfy certain types of **consistency constraints**. Integrity constraints (e.g. account balance > 0) become part of program code rather than being stated explicitly.

- Hard to add new constraints or change existing ones.

5. Atomicity problems

- Failures may leave database in an inconsistent state with partial updates carried out. E.g., transfer of funds from one account to another should either complete or not happen at all – so it must be atomic. It is difficult to ensure atomicity in conventional file-processing system.

6. Concurrent-access anomalies

- Needed for system performance and faster response.

- Uncontrolled concurrent accesses can lead to inconsistencies. E.g. two people reading a balance and updating it at the same time – so supervision is needed.

7. Security problems

- Not every user of the database system should be able to access all the data. ---

- This is hard to provide user access to some data, but not all.

Database systems offer solutions to all these problems

1.3 View of Data

A database system is a collection of interrelated data and set of programs that allow users to access and modify these data. A major purpose of a database system is to provide users an **abstract view** of the data. That is, the system hides certain details of how the data is stored and maintained.

1.3.1 Data Abstraction

Several levels of data abstraction are maintained to simplify users' interaction with the system:

1. Physical level: The lowest level of abstraction describes **how** data are actually stored. It describes complex low-level data structures in detail.

2. Logical level: The next higher level of abstraction describes **what** data stored in database, and what relationships exist among those data. The logical level thus describes the entire database in terms of a number of relatively simple structures. For Example:

```
create table instructor (
        ID varchar (5),
        name varchar (20),
        dept_name varchar (20);
        salary number (8,2));
```

Although implementation of the simple structures at the logical level may involve complex physical-level structures, the user of the logical level does not need to be aware of this complexity. This is referred to as **physical data independence**. Database administrators, who must decide what information to keep in the database, use the logical level of abstraction.

3. View level: The highest level of abstraction describe only part of the entire database. Although the logical level uses simpler structures, complexity remains because of the variety of the information stored in a large database. Many users of the database system do not need all this information; instead they need to access only a part of the database. The view level of abstraction exists to simplify their interaction with the system. The system may provide many views for the same database. In addition to hiding details, the views also provide a security mechanism to prevent users from accessing certain part of the database.

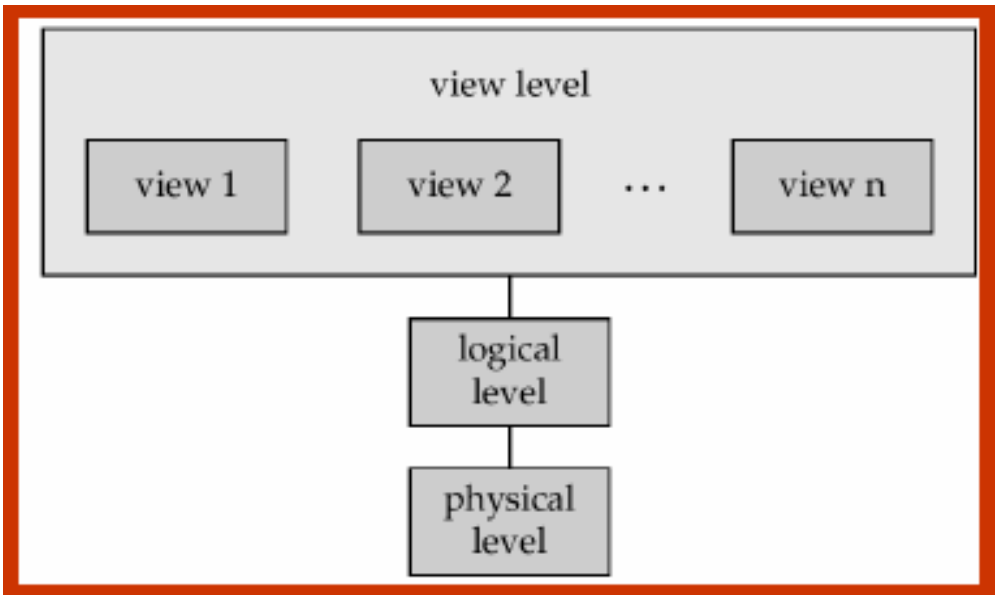


Figure 1.1 The three levels of data abstraction

1.3.2 Instances and Schemas

Schema: The overall design of the database is called the database schema. Schemas are changed infrequently, if at all. This corresponds to the variable declaration (with type definition) of a programming language. Schemas are:

1. Physical schema: Describes the database design at physical level

2. Logical schema: Describes the database design at logical level

3. Subschemas: A database may also have several schemas at the view level, sometimes called **subschemas**, that describe different views of the database.

Instances: Databases change over time as information is inserted and deleted. The collection of information stored in the database at a particular moment is called an instance of the database. While executing, each variable of a programming language has a particular value at a given instant. The values of the variable in a program at a point in time correspond to an instance of the database schema.

Physical Data Independence – Of these, the logical schema is by far the most important, in terms of its effect on application programs, since programmers construct applications by using the logical schema. The physical schema is hidden beneath the logical schema, and can usually be changed easily without affecting application programs. Application programs are said to exhibit **physical data independence** if they do not depend on the physical schema, and thus need not be rewritten if the physical schema changes.

1.3.3 Data Models

A data model is a collection of conceptual tools for describing data, data relationship, data semantics and consistency constraints. A data model provides a way to describe the design of a database at the physical, logical and view level.

❖ Some Data Models

1. Relational Model
2. Entity-Relationship (E-R) Data Model
3. Object-Based Data Model – i) Object-Relational Model
   ii) Object-Oriented Model
4. Semi-structured Data Model
5. Network Data Model – not used currently
6. Hierarchical Data Model – not used currently

1. Relational Model: This is a lower level model. It uses a collection of tables to represent both data and relationships among those data. The relational model is the most widely used data model and a vast majority of current database systems are based on the relational model.

- Each table has multiple columns, and each column has a unique name.

- The relational model is an example of a record-based model. This is because the database is structured in fixed-format records of several types. Each table contains records of a particular type. Each record type defines a fixed no. of fields, or attributes. The columns of the table correspond to the attributes of the record type.

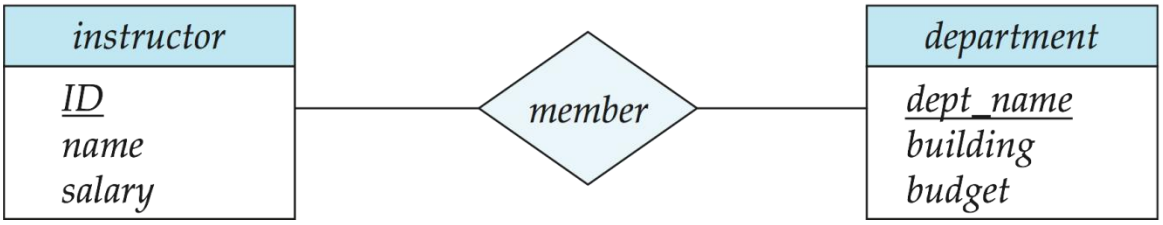
ID	*name*	*dept_name*	*salary*
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

(a) The *instructor* table

dept_name	*building*	*budget*
Comp. Sci.	Taylor	100000
Biology	Watson	90000
Elec. Eng.	Taylor	85000
Music	Packard	80000
Finance	Painter	120000
History	Painter	50000
Physics	Watson	70000

(b) The *department* table

2. Entity-Relationship Model: This is a higher-level data model. It is based on a perception of a real world that consists of a collection of basic objects, called *entities* and the *relationship* among these objects. An entity is a "thing" or "object" in the real world that is distinguishable from all other objects. The entity-relationship model is widely used in database design.



3. Object-Based Data Model:

i) Object-oriented data model: Object-oriented programming (especially in Java, C++, or C#) has become the dominant software-development methodology. This led to the development of an object-oriented data model that can be seen as extending the E-R model with notions of encapsulation, methods (functions), and object identity.

ii) Object-relation data model: Combines the features of **object-oriented data model** and **relational data model.**

4. Semistructured data model: Permits the specification of data where individual data items of the same type may have different sets of attributes. The extensible markup language (XML) is widely used to represent semistructured data.

Historical Models: precede the relational data model. These are in little use now.

i) Network data model

ii) Hierarchical data model

1.4 Database Languages

SQL (Structured Query Language) – Widely used database language

Parts of SQL:

1.4.1 Data-Manipulation Language (DML):

- DML is a language that enables users to access or manipulate data as organized by appropriate data model. The types of accesses are:

i) The retrieval of information stored in the database - Query

ii) The insertion of new information into the database - insert

iii) The deletion of information from the database - delete

iv) The modification of information stored in the database - update

Classes of DML: There are basically two types:

1. Procedural DMLs – user specifies **what** data are required and **how** to get or compute the data. e.g. Relational Algebra

2. Nonprocedural DMLs (Declarative DMLs) – user specifies **what** data are required **without** specifying how to get or compute the data. e.g. SQL

- Declarative DMLs are usually easier to learn and use than procedural DMLs. However, since a user does not have to specify how to get the data, the database system has to figure out an efficient means of accessing data.

Query:

A **query** is a statement requesting the retrieval of information. The portion of a DML that involves information retrieval is called **a query language**.

Examples:

1. Commercially used: SQL
2. Experimentally used: QBE (Query-by-example) and Datalog

General format of SQL query:

select A1, A2, A3
from r1, r2, r3
where P

Example1: Find the name of the instructor with ID 22222

Select name
from instructor
where instructor.ID **=** '22222'

Example2: Find the ID and building of instructors in the Physics dept.

```
select instructor.ID, department.building
from instructor, department
where instructor.dept_name = department.dept_name and
        department.dept_name = 'Physics'
```

1.4.2 Data-Definition Language (DDL):

- A database schema is specified by a set of definitions expressed by a special language called a **data-definition language**. The DDL is also used to specify additional properties of data.

```
create table instructor (
                ID varchar (5),
                name varchar (20),
                dept_name varchar (20);
                salary number (8,2));
```

1. Data storage and definition language:

The storage structure (B+ tree, B-tree, Heap, Hash) and access methods (sequential, Binary, Indexing, Hashing) used by the database system are specified by a set of statements in a special type of DDL called data storage and definition language. These statements define the implementation details of the database schemas, which are usually hidden from the users.

2. Consistency constraints specification:

The data values stored in the database must satisfy certain consistency constraints. The DDL provides facilities to specify such constraints. The database system checks these constraints every time the database is updated.

i) Domain constraints: A domain of possible values must be associated with every attribute (integer types, character types, date/time types). Declaring a attribute to be of a particular domain acts as a constraint on the values that it can take. They are easily tested by the system whenever a new data is entered into the database.

ii) Referential integrity: There are cases where it is to ensure that a value that appears in one relation for a given set of attributes also appears for a certain set of attributes in another relation (referential integrity). Database modification can cause violations of referential integrity. When it is violated, the normal procedure is to reject the action that caused the violation.

iii) Assertions: An assertion is any condition that the database must always

satisfy. Domain constraints and referential integrity constraints are special forms of assertions. However there are many constraints for which we have to seek help of assertions. For example, “Every department must have at least five courses offered every semester” must be expressed as an assertion. When an assertion is created, the system tests it for validity. If the assertion is valid, then any future modification to the database is allowed only if it does not cause that assertion to be violated.

iv) Authorization: It is required to differentiate among the users as far as the types of access they are permitted on various data values in the database. These differentiations are expressed in terms of authorizations, the most common instance related authorization being: **read** (reading but not modification)**, insert** (but not modification of existing data)**, update** (but not deletion)**, delete**. A user may be assigned all, none or a combination of these types of authorization. Other schema related authorizations might be: **resource** (create), **alter** (object structure modification **alter-add** and **alter-modify**), **drop** (delete object), **rename** (object or component of object) etc.

3. Data dictionary or Data Directory:

As DDL creates schema of a table it also updates a special set of tables called data directory or data dictionary. A data dictionary contains **metadata** (data about data). The data dictionary is considered to be a special type of table, which can only be accessed and updated by the database system itself (not a regular user). A database system consults the data dictionary before reading or modifying actual data.

1. Relation-metadata (relation-name, no-of-attributes, storage-organization, location)
2. Attribute-metadata (attribute-name, relation-name, domain-type, position, length)
3. User-metadata (user-name, encrypted-password, group)
4. Index-metadata (index-name, relation-name, index-type, index-attributes)
5. View-metadata (view-name, definition)

1.12 Database Users and Administrators

A primary goal of a database system is to retrieve information from and store new information in the database. People who work with a database can be categorized as database users or database administrators.

1.12.1 Database Users

Users are differentiated by the way they expect to interact with the system. Four different types:

1. Naive users – are unsophisticated users who interact with the system by invoking one of the permanent application programs that have been written previously.

E.g. people accessing database over the web, bank tellers, and clerical staff.

The typical user interface for naïve users is a forms interface, where the user can fill in appropriate fields of the form. They may also simply read reports generated from the database.

2. Application programmers – are computer professionals who write application programs. Application programmers can choose from many tools to develop user interface. **Rapid application development (RAD)** tools are tools that enable an application programmer to construct forms and reports with minimal programming effort.

3. Sophisticated users – interact with the system without writing programs. Instead, they form their requests in a database query language. Analysts who submit queries to explore data in the database fall in this category.

e.g., analyst looking at sales data (OLAP – Online analytical processing), data mining

4. Specialized users – are sophisticated users who write specialized database applications that do not fit into the traditional data processing framework.

e.g., computer-aided design systems, knowledge-base and expert systems and environment-modeling systems – uses complex data types.

1.12.2 Database Administrator

One of the main reasons for using DBMS is to have central control both data and the programs that access that data. A person who has such central control over the system is called **database administrator (DBA)**.

The functions of a database administrator (DBA) include:

1. Schema definition: The DBA creates the original database schema by executing a set of data definition statements in the DDL.

2. Storage structure and access method definition: File organization (sequential, heap, hash, B^+ tree), organization of records in a file (fixed length or variable length), index definition (ordered index, hash index).

3. Schema and physical-organization modification: The DBA carries out

changes to the schema and physical organization to reflect the changing needs of the organization, or to alter the physical organization to improve the performance.

4. Granting of authorization for data access: By granting different types of authorization, the DBA can regulate which parts of the database various users can access.

5. Specifying integrity constraints: The DBA implements key declaration (primary key, foreign key), trigger, assertion, business rules of the organization.

7. Acting as liaison with users

8. Routine maintenance:

i) Periodically backing up the database, either onto tapes or remote servers, to prevent loss of data in case of disasters

ii) Ensuring that enough disk space is available for normal operations and upgrading disk space as required

iii) Monitoring jobs running on the database and ensuring better performance

1.7 Database Storage and Querying

A database system is portioned into modules that deal with each of the responsibilities of the overall system. The functional components of a database system can be broadly divided into **the storage manager** and **the query processor components**.

1.7.1 Storage Manager

A storage manager is a program module that provides the interface between the low-level data stored in the database and the application programs and queries submitted to the system.

- The storage manager **components** include:

1. Authorization and integrity manager: tests integrity constraints and checks the authority of users to access data.

2. Transaction manager: ensures consistency of the DB despite system failures.

3. File manager: allocation of space + data structure used to represent

information stored in the disk.

4. Buffer manager: responsible for fetching data from disk storage into main memory, and deciding what data to cache in the main memory.

- The storage manager implements several data structures as part of the physical system implementation:

1. Data files: which store the database itself (relations).

2. Data dictionary: stores the metadata about the structure of the database (sometimes called catalog).

3. Indices: provide fast access to data items. Like the index of a text book, a database index provides pointers to those data items that hold a particular value.

1.7.2 The Query Processor

Components:

1. DDL interpreter: interprets DDL statements and records the definitions in the data dictionary.

2. DML compiler: translates DML statements in a query language into an evaluation plan consisting of low-level instructions that the query evaluation engine understands.

3. Query evaluation engine: executes low-level instructions generated by the DML compiler.

1.8 Transaction Management

A **transaction** is a collection of operations that performs a single logical function in a database application.

_ e.g., deposit, withdrawal, transfer between accounts

- **ACID** Properties of Transaction:

A - Atomicity
C - Consistency
I - Isolation
D – Durability

- **Transaction Manager** = concurrency control manager + recovery manager

- **Concurrency-control manager** controls the interaction among the concurrent transactions, to ensure the consistency of the database.

  _ e.g., two users accessing the same bank account cannot “corrupt” the system or withdraw more than allowed

- **Recovery Manager** ensures that the database remains in a consistent (correct) state despite system failures (e.g., power failures and operating system crashes) and transaction failures.

_ e.g., system crash cannot wipe out “committed” transactions

1.9 Database Architecture

It can be provide a single picture of various components of a database system and the connection among them.

The architecture of the database greatly influenced by the underlying computer system on which the database system runs. Database systems can be **centralized** or **client-server**, where the server machine executes work on behalf of multiple client machines. Database system can also be designed to exploit parallel computer architecture. Distributed database span multiple geographically separated machines.

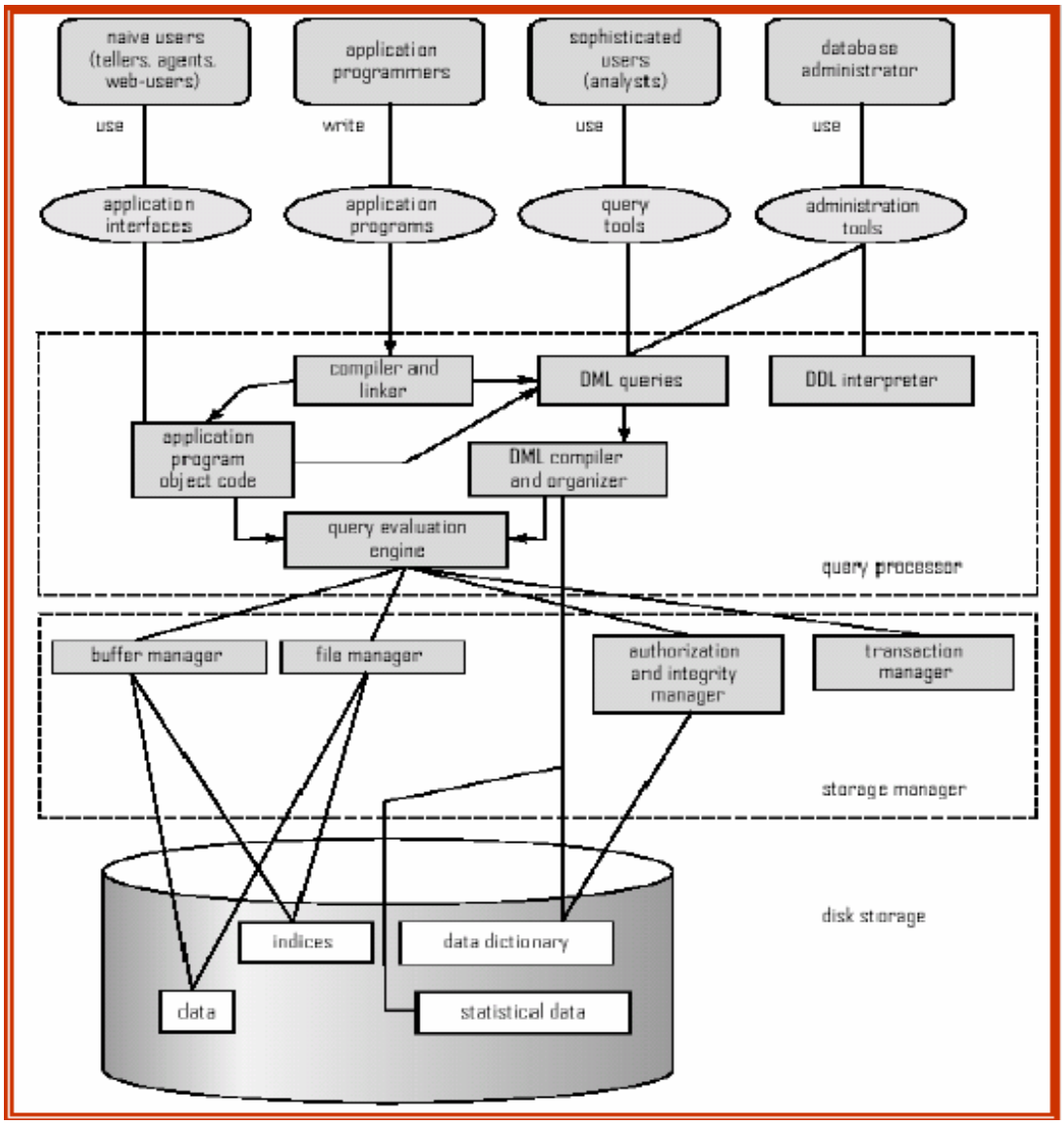


Figure 1.5: Overall System Structure

Most user of a database system today are not present at the site of the database system, But connect to it through a network. So we can differentiate between **client** machines, on which remote database users work, and **server** machines, on which the database systems runs.

Database applications are usually partitioned into two or three parts:

1. **Two-tier architecture:** The application is partitioned into a component that resides at the client machine, which invokes database system functionality at the server machine through query language statements. Application program interface standard like ODBC and JDBC are used for interaction between client and server.

2. **Three-tire architecture:** The client machine acts merely a front end and does not contain any direct database calls. Instead, the client end communicates with **an application server**, usually through a forms interface. The application server in turn communicates with a database system to access data. Three-tire applications are more appropriate large applications that run on the World Wide Web.

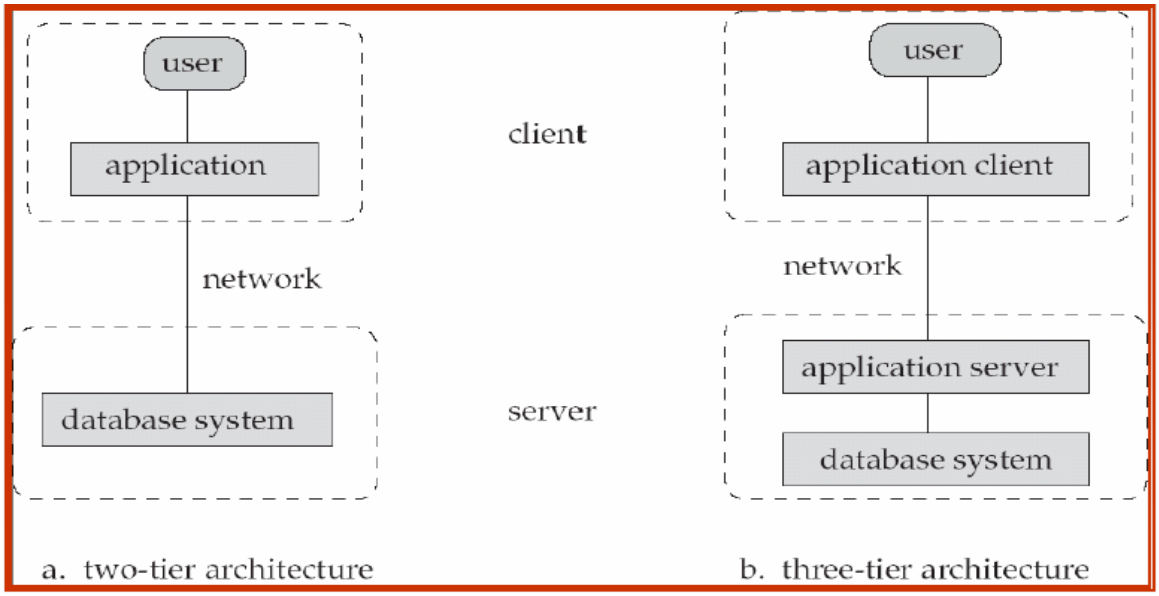


Figure 1.6: Two-tire and three-tire architectures

1.10 Data Mining and Information Retrieval

The term data mining refers loosely to the process of semiautomatically analyzing large database to find useful patterns. Like knowledge discovery in artificial intelligence (also called **machine learning**) or statistical analysis, data mining attempts to discover rules and patterns from large volume of data, stored primarily in disk. So data mining deals with “knowledge discovery in database (KDD).

Usually there is a manual component to data mining, consisting of preprocessing data to a form acceptable to the algorithms and post-processing of discovered patterns to find novel ones that could be useful with the user defined parameters of “support” and “confidence”. There may also be more than one type of pattern that can be discovered from a given database and manual interaction may be needed to pick useful types of patterns.

Here is the end of Introduction